Version: 15122023

Data Analysis with Databricks SQL

Learning Objective

- Describe how Databricks SQL:
  - Works in the Databricks Lakehouse architecture
  - Implements data Security
- Use Databricks SQL to:
  - Ingest data into the Lakehouse
  - Use SQL commands to CREATE objects
  - Query data
  - Create Visualizations and Dashboards
  - Share queries and dashboards with others
  - Use automation (Alerts) and integration capabilities

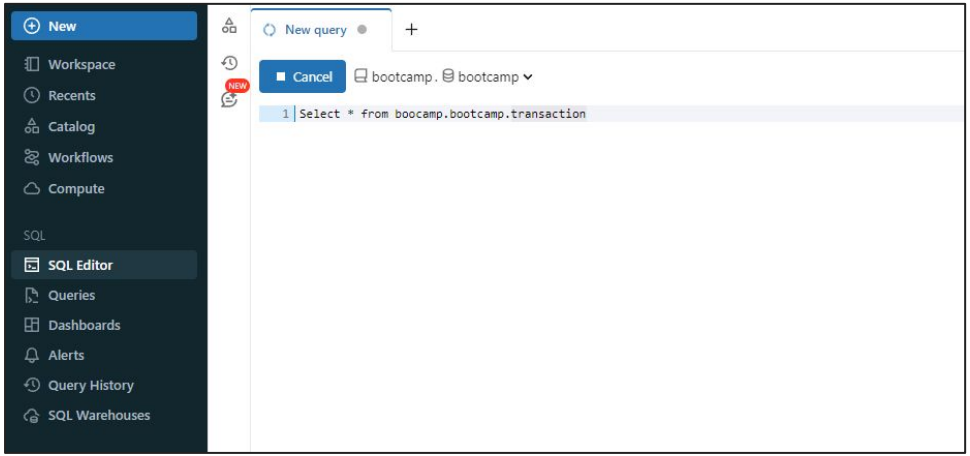
Navigating Databricks SQL

SQL Editor

1. Type below code into your 'New Query' tab
2. Press Run
3. Name your Query from 'New Query' to 'Your Name'
4. Click Save button

<u>Code:</u>

Select * from bootcamp.bootcamp.transaction

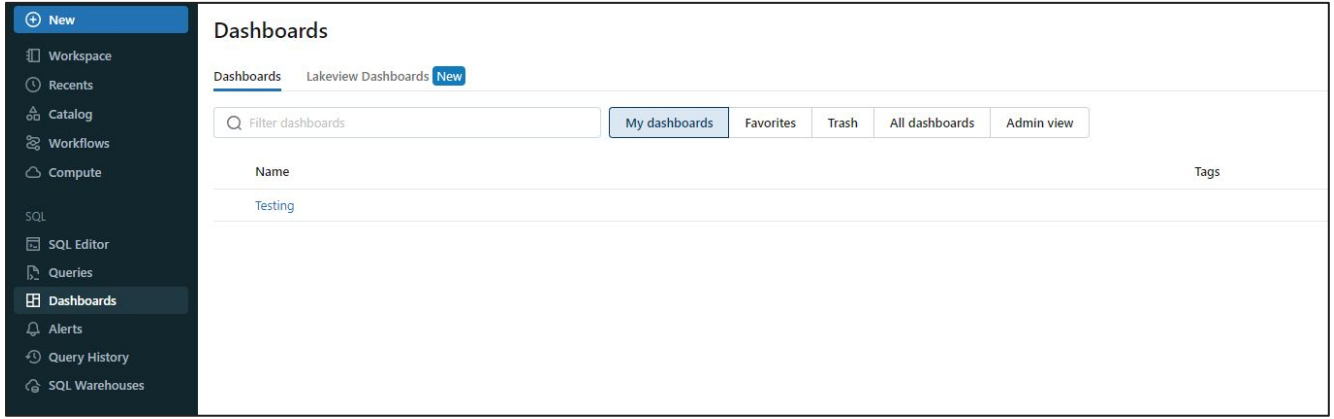


Queries tab

1. Check if your saved query is in Queries tab
2. You can search, open, make favorite or delete the query

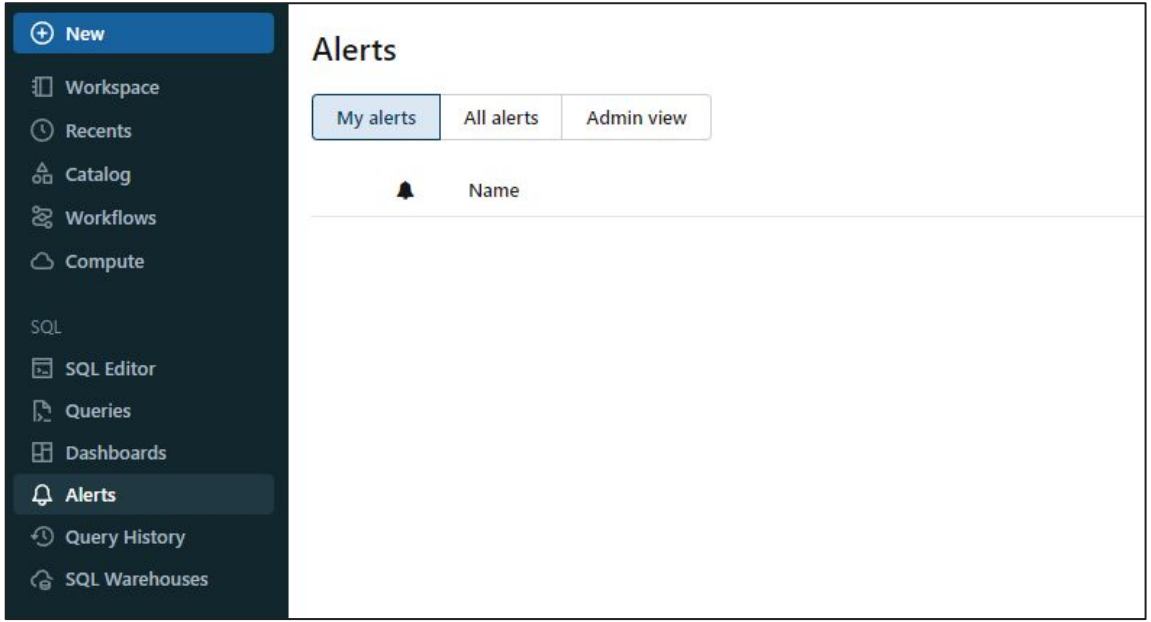
Dashboard tab

- You can search, open, make favorite or delete the dashboard



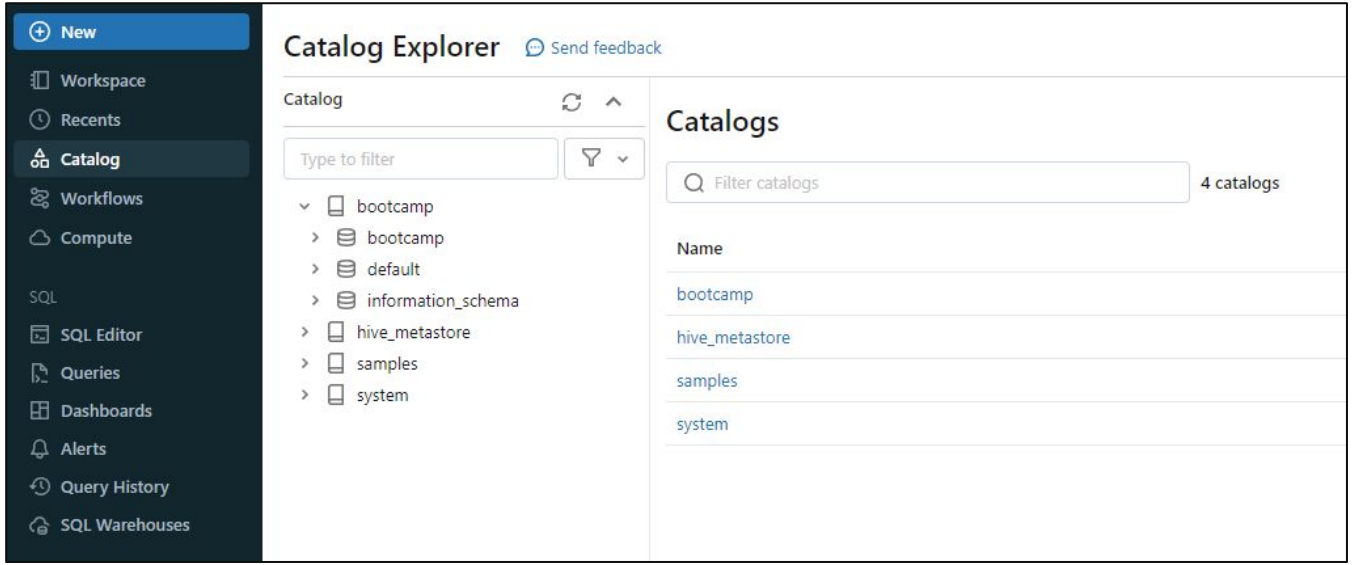
Alerts tab

- Alerts notify you via email when a field returned by a scheduled query meets a threshold



Catalog Explorer tab

- Catalog Explorer lets you explore and manage permissions on catalogs/schemas/tables/views

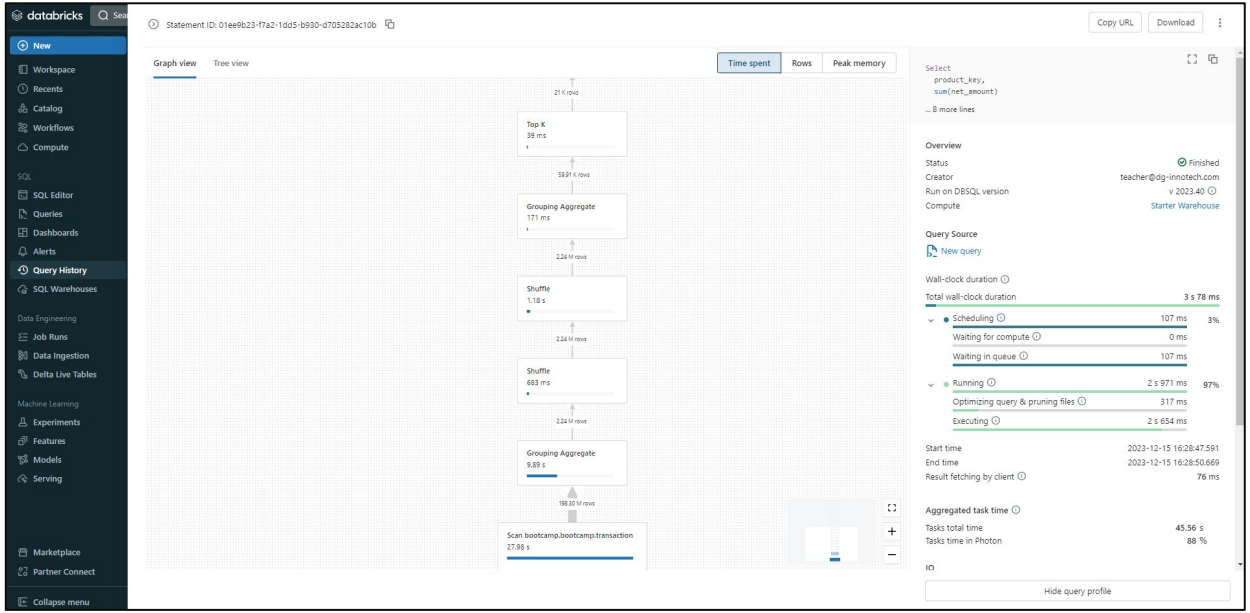


Query History

1. Run below code in 'SQL Editor'
2. Go to 'Query History'

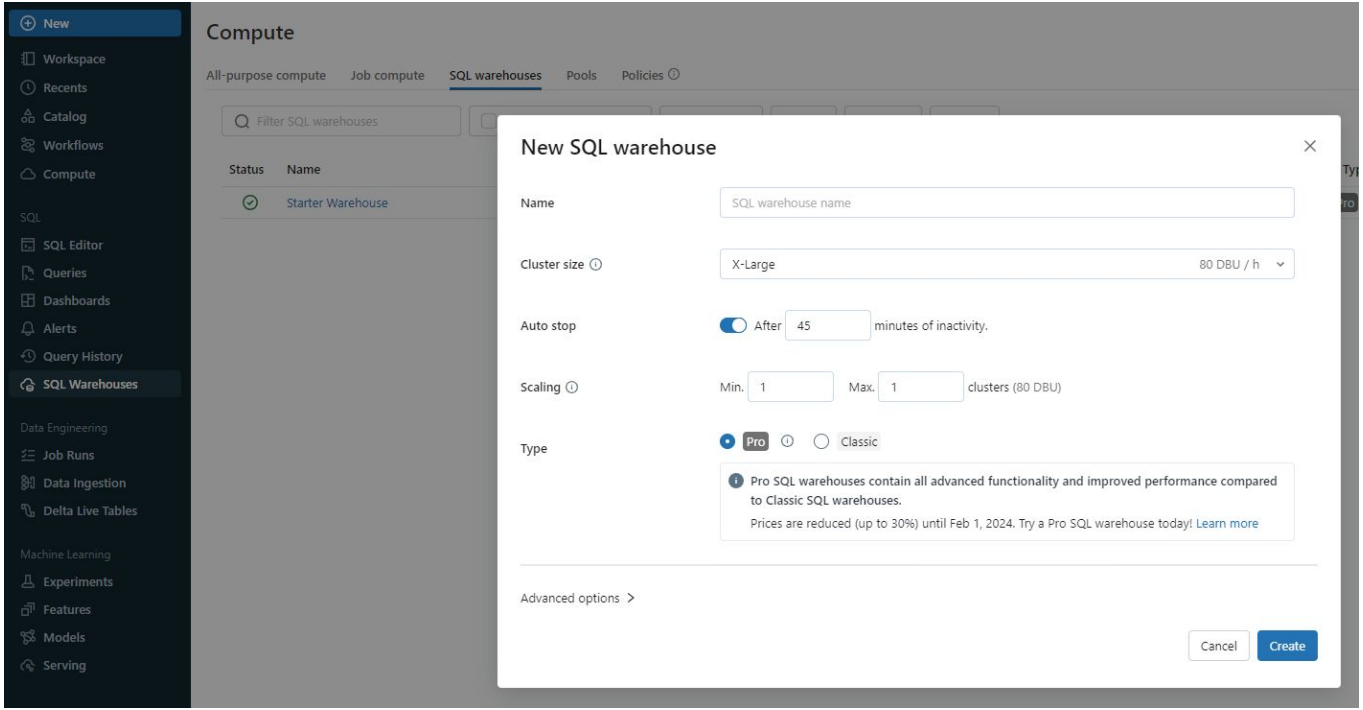
<u>Code:</u>

```
Select product_key, sum(net_amount)
from bootcamp.bootcamp.transaction
group by product_key
order by product_key;
```



SQL Warehouse

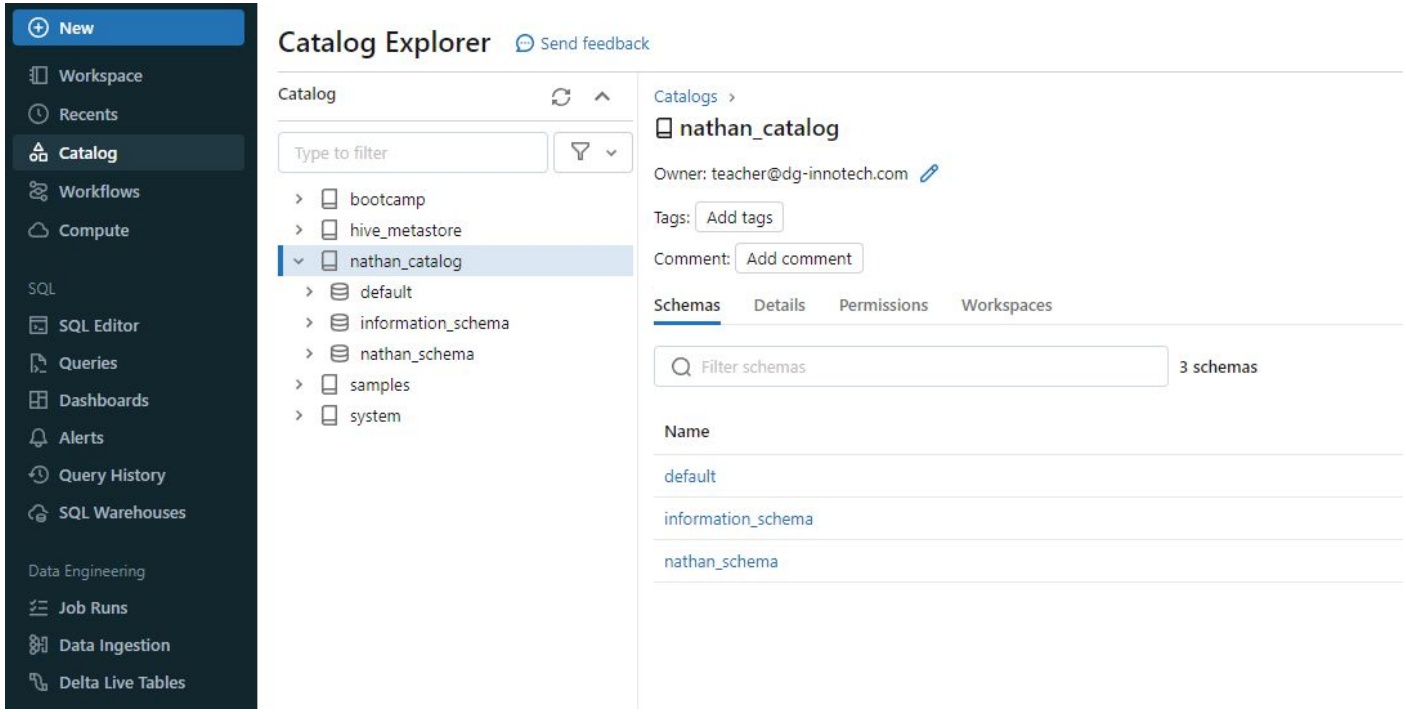
- a compute resource that lets you run SQL commands on data objects within Databricks SQL



Setting Up a Catalog and Schema

Setting Up a Catalog and Schema

1. Create a Catalog (to hold your Schemas)
2. Create a Schema (to hold Tables/Views/Functions)



Data Importing

Create Table

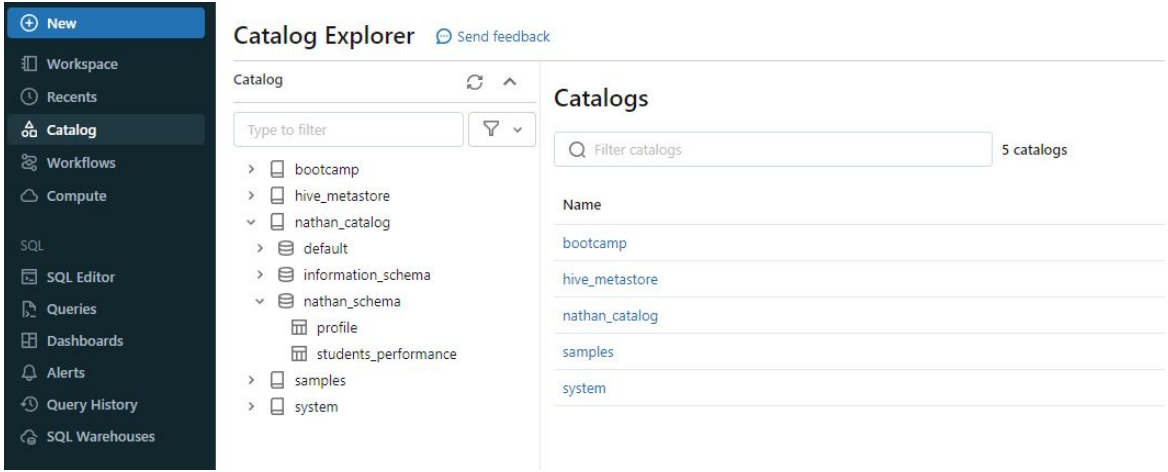
1. Upload a data file
2. Create a table from the file
3. Create a new Tables from existing Table with below code
4. Use 'Catalog Explorer' tab to GRANT/REVOKE permissions to Table

Code:

Create table profile as
Select * from nathan_catalog.nathan_schema.students_performance

Data:

https://www.kaggle.com/datasets/spscientist/students-performance-in-exams

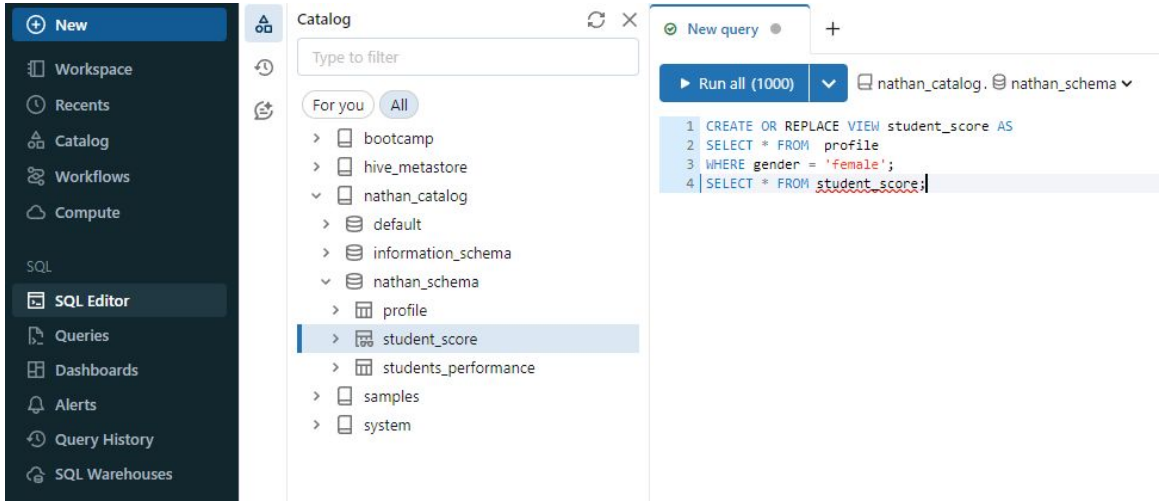


Create View

1. Create view with below code in SQL Editor

Code:

```
CREATE OR REPLACE VIEW student_score AS
SELECT * FROM  profile
WHERE gender = 'female';
SELECT * FROM student_score;
```

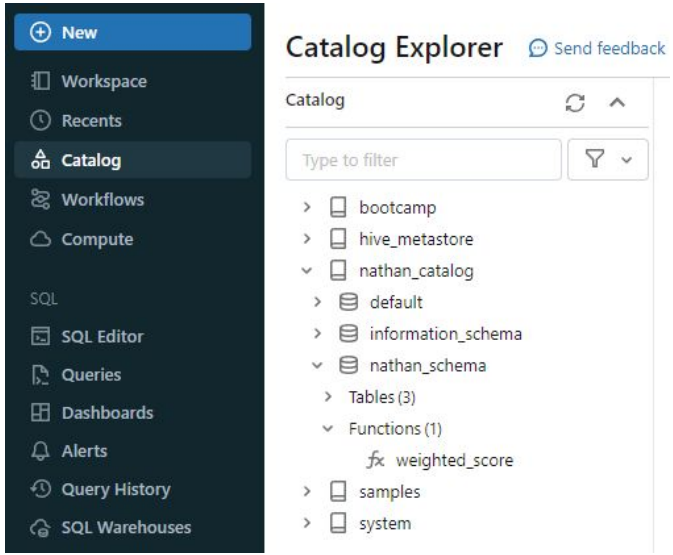


Create Function

1. Create function with below code in SQL Editor
2. Check if the function is under Catalog Explorer

Code:

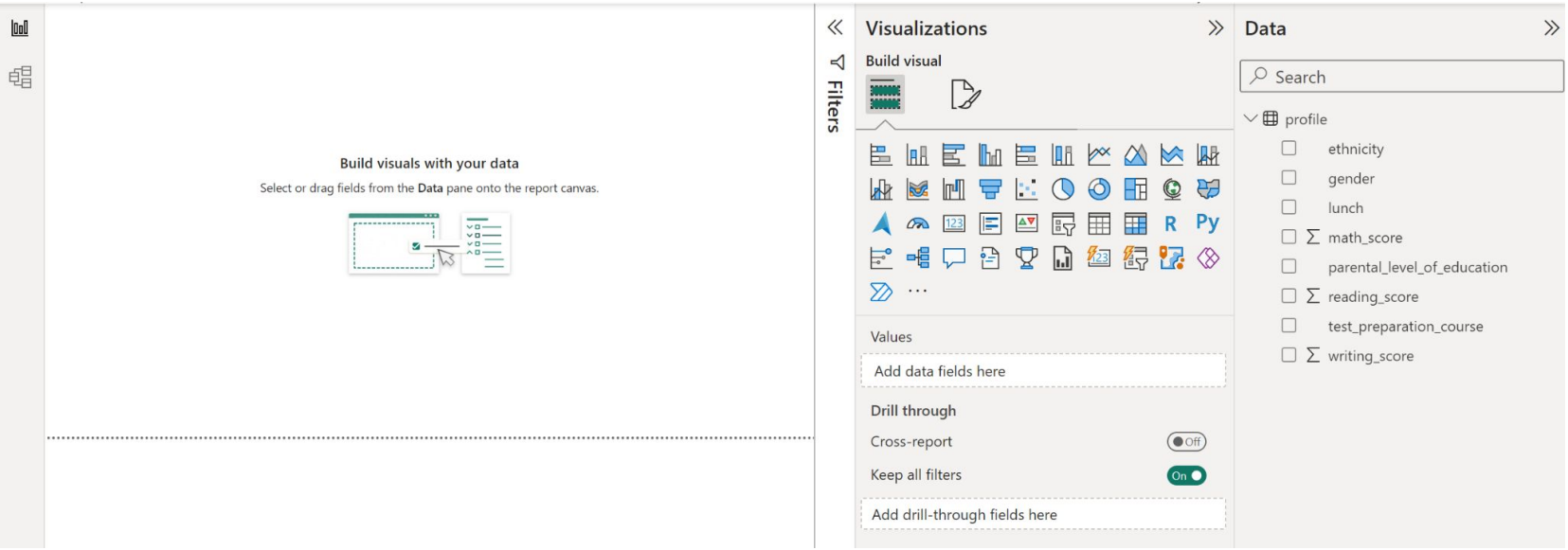
```
CREATE FUNCTION weighted_score (score DOUBLE)
RETURNS DOUBLE RETURN score*0.3;
SELECT weighted_score(100);
```



Integration

Connect with external BI tool

1. Create Personal Access Token in User Setting
2. Connecting to Power BI with Partner Connect



Delta Lake in Databricks SQL

Create Schema with One Table

1. Run the code below in SQL editor

Code:

```
DROP SCHEMA IF EXISTS temporary_schema CASCADE;
CREATE SCHEMA IF NOT EXISTS temporary_schema;
CREATE OR REPLACE TABLE temporary_schema.simple_table (width INT, length INT, height INT);
INSERT INTO temporary_schema.simple_table VALUES (3, 2, 1);
SELECT * FROM temporary_schema.simple_table;
```

2. Run the code below to see information about the table we just created

Code:

```
DESCRIBE EXTENDED temporary_schema.simple_table;
```

3. Run the code below to drop the table

Code:

```
DROP TABLE IF EXISTS temporary_schema.simple_table;
```

Time Travel

1. Run the code below to see the log of changes

Code:

```
DESCRIBE HISTORY nathan_catalog.nathan_schema.profile;
```

2. Run the code below to make two seperate changes to the table

Code:

```
UPDATE nathan_catalog.nathan_schema.profile SET gender = 'F' WHERE gender = 'female';
UPDATE nathan_catalog.nathan_schema.profile SET gender = 'M' WHERE gender = 'male';
DESCRIBE HISTORY nathan_catalog.nathan_schema.profile;
```

3. Run the code below to select specific version of the table

Code:

```
SELECT gender FROM nathan_catalog.nathan_schema.profile VERSION AS OF 1;
```

4. Run the code below to restore a table to a past state

Code:

```
RESTORE TABLE nathan_catalog.nathan_schema.profile TO VERSION AS OF 1;
DESCRIBE HISTORY profile;
```

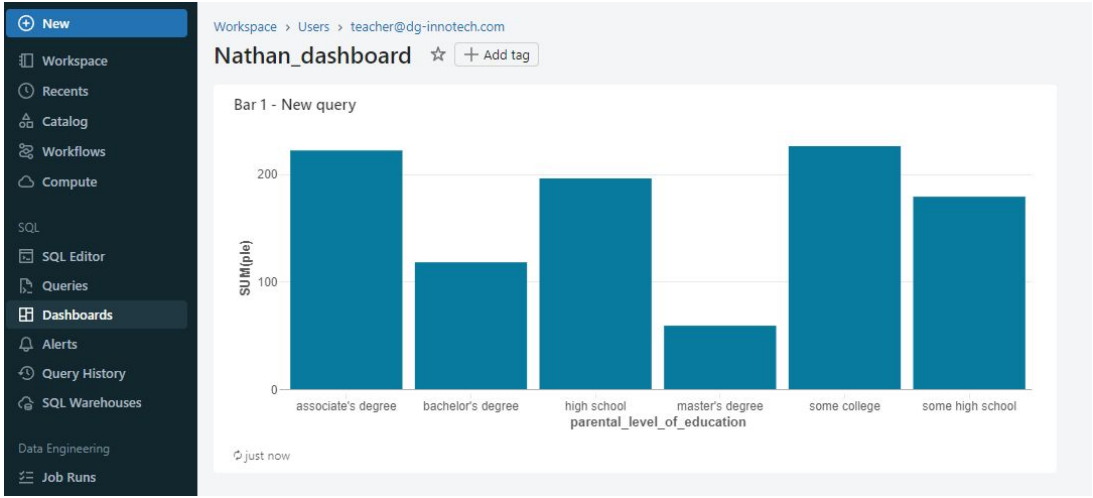
Data Visualization and dashboard

Bar Chart

1. Query data
2. Create a Visualization
3. Create a Dashboard

<u>Code:</u>

```
Select parental_level_of_education,
count(parental_level_of_education) as ple from profile
group by parental_level_of_education
order by ple desc;
```



The Counter

1. Open a new query tab and run the code below

Code:

```
SELECT avg(math_score) AS average_math_score, 90 AS score_goal
FROM nathan_catalog.nathan_schema.profile;
```

2. Save the query as "Average Math Score"
3. In the query results section, hover over the "+" symbol, and click "Visualization"
4. Select "Counter" as the visualization type
5. For "Counter Label" type "Average Math Score"
6. For "Counter Value Column" make sure the column "average_math_score" is selected
7. For "Target Value Column" choose "score_goal"
8. Add to Dashboard

Create a Boxplot

1. Open a new query tab and run the code below

Code:

```
SELECT * FROM nathan_catalog.nathan_schema.profile;
```

2. Save the query as "Boxplot"
3. In the query results section, hover over the "+" symbol and click "Visualization"
4. Select "Box" as the visualization type
5. For "X Column" select "parental_level_of_education"
6. For "Y Column" select "reading_score"
7. Click the "X Axis" tab
8. Enter "education level" in the "Name" field to make more user friendly
9. Click the "Y Axis" tab
10. Enter "score" in the "Name" field to make it more user friendly
11. Add to Dashboard

Parameterized Queries

Writing a Query with Parameters

1. Start a new query in SQL Editor
2. Paste the query below into the editor and save the query as "Education Level"

Code:

```
SELECT DISTINCT parental_level_of_education FROM nathan_catalog.nathan_schema.profile;
```

3. Start a new query, and paste the code below in the editor

Code:

```
SELECT avg(math_score) as average_math FROM nathan_catalog.nathan_schema.profile
where parental_level_of_education = ' ';
```

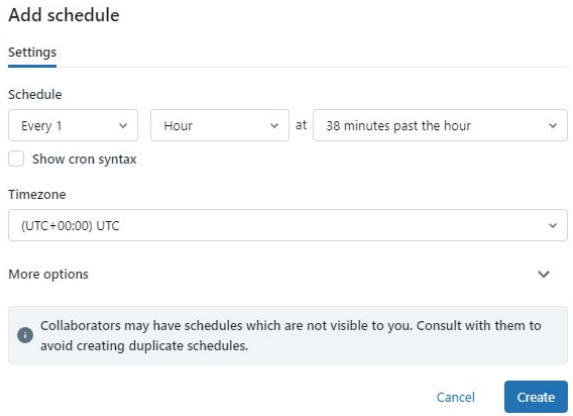
4. Place your cursor in-between the single quotes, hover over the "+" symbol, and click "Parameter"
5. Input "Education Level" for the "Keyword" field
6. Drop down "Type" and choose "Query Based Dropdown List"
7. For "Query" choose the query we created above: "Education Level"
8. Click "Add Parameter"
9. Save the query as "Average Math Score"
10. Open the dropdown list and choose a Category from the list
11. Click "Apply Changes"

Databricks SQL in Production

Query Refresh Schedule

1. Choose a query in SQL Editor
2. Click on "Schedule" next to "Save"
3. Click "Add Schedule" from the dropdown
4. Configure the schedule for every hour and confirm your timezone from the dropdown
5. Click “Create” to confirm the schedule for the query

Alert: If the refresh rate is less than the SQL Warehouse 'Auto Stop' parameter, **the Warehouse will run indefinitely, impacting platform costs.**

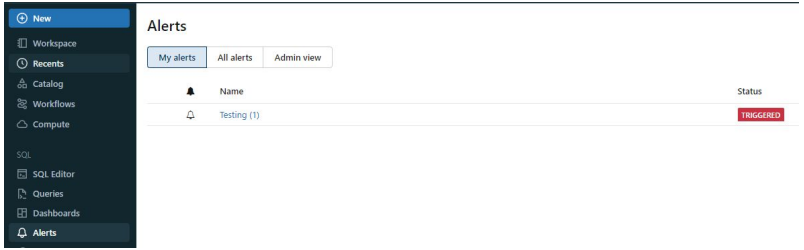


Alerts

1. Click "Alerts" in the sidebar menu
2. Click "Create Alert"
3. On the resulting screen, name your alert
4. From the “Query” dropdown, select the query made earlier
5. From the Trigger Condition option, select "count(1) First row" in the "Value column" and modify the "Threshold value" to " > 1000"
6. Click “Create Alert” at the bottom
7. Run the code below to add data
8. Refresh the alert to trigger

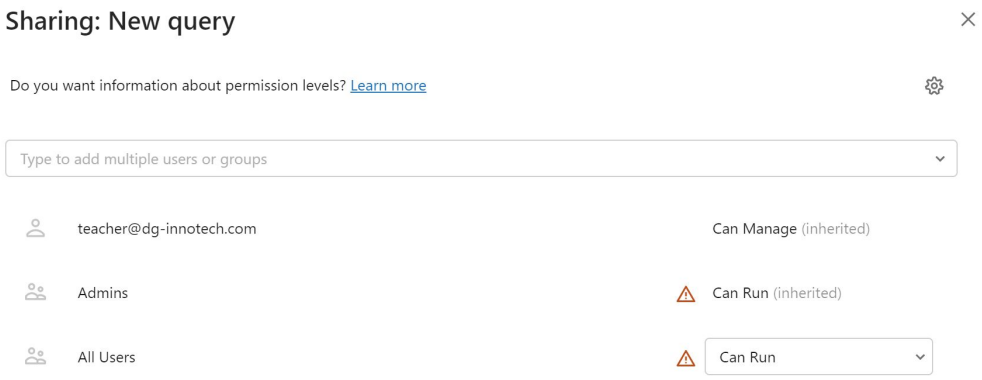
Code:

```
INSERT INTO nathan_catalog.nathan_schema.profile
(gender,ethnicity,parental_level_of_education,lunch,test_preparation_course,math_score,reading_score,writing_score)
VALUES ("F","group A","some college","standard","test_preparation_course",100,100,100);
```



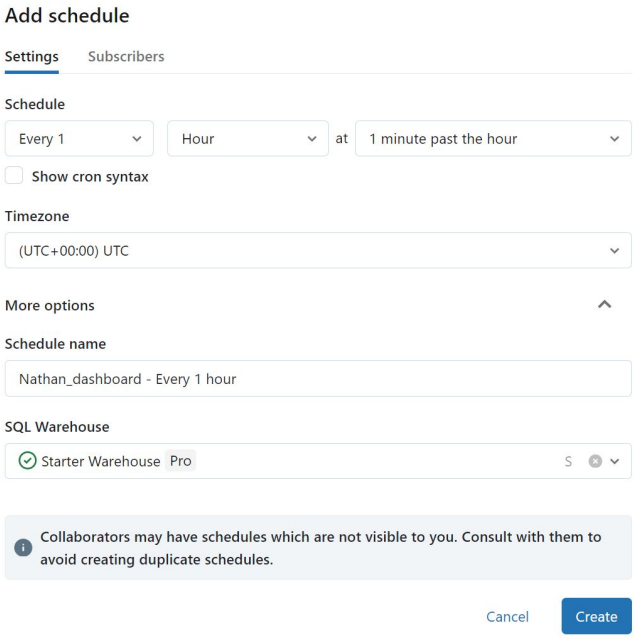
Sharing Queries

1. Select "SQL Editor" from the sidebar menu
2. Choose the query tab that you want to share from the top of the SQL Editor window
3. Select "Share" from the upper right corner
4. Select the dropdown box that reads "Type to add multiple users or groups." You will be prompted with suggestions, but instead, you can type a user or group name to make your selection. Choose "All Users" for this example
5. Toggle the option for “Can Manage” to “Can Edit,” “Can Run,” or “Can View.”
6. Click “Add” to close the dialoge box.
7. Close the Sharing dialogue by clicking the "X" in the upper right corner.



Refreshing Dashboards and Sharing Results

1. Click "Schedule" in the upper right corner of the dashboard you want to schedule for refresh
2. Indicate the frequency at which you want the dashboard to refresh
3. Set the timezone for your schedule if necessary
4. Under "More options," you can edit the schedule name and select a SQL Warehouse that will be used to refresh the dashboard
5. Select “Subscribers” next to “Settings” in the “Add Schedule”
6. Use the "Search to add users or destinations" field to add users, groups, or destinations to the list of subscribers for this refresh schedule
7. After completing the configuration, click "Create" to set the schedule



Warning: If the Dashboard refresh interval is less than the SQL Warehouse 'Auto Stop' parameter, **the Warehouse will run indefinitely, impacting platform costs.**